

A language for websites — and the paper they print on.

A file describes what a page shows: its elements, the state behind them, what happens when the reader acts. The compiler turns it into a static site, an app, a PDF or a deck.

```
terminal

$ wf init hello --template spa
Created hello/ with AGENTS.md
$ wf build
Building hello...
app.js 4.6 kB gzipped
0 errors, 0 warnings
$ wf build -d deck
Slides: 16 slide(s)
```

static site

SPA

PDF

slides

custom elements

THE IDEA

One file is the page.

WHAT YOU WRITE

Home.wf

```
src/pages/Home.wf

page Home(path: "/", title: "Home") {
  state count = 0
  derived doubled = count * 2

  Container {
    Heading("Welcome").h1
    Text("Count: {count} · doubled {doubled}")
    Button("+1").primary {
      on click { count = count + 1 }
    }
  }
}
```

WHAT THE READER GETS

index.html · app.js



localhost:3000/

Welcome

Count: 3 · doubled 6

3 +1

The text repaints when the state changes — no virtual DOM, no hooks, no build config.

01

The language

Declarations are lowercase, elements capitalised, and anything that resolves to nothing is an error — never a silent no-op.



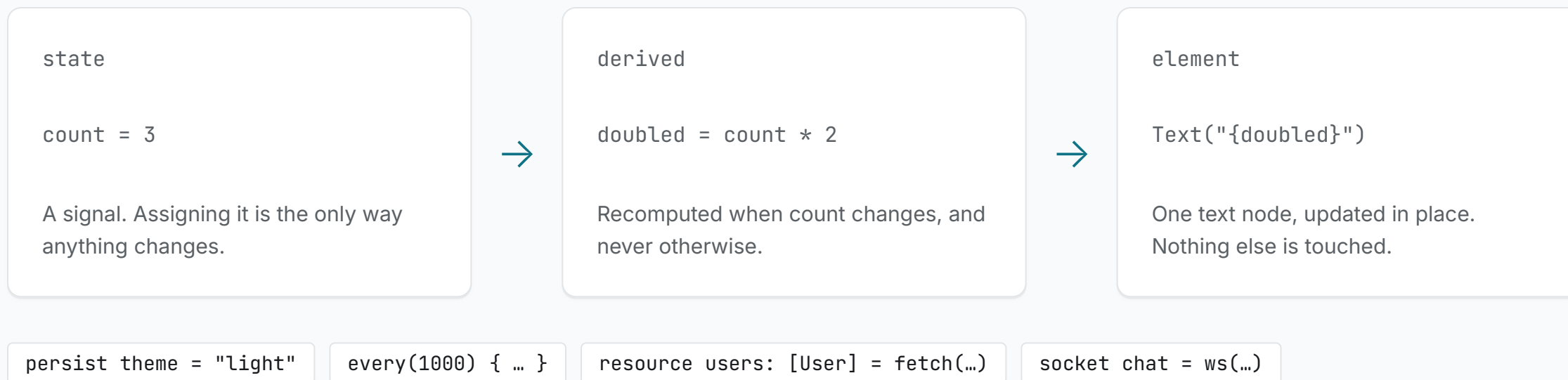
One lexeme, one meaning.

Seven shapes cover everything a page says. The compiler, the linters and the editor read one registry of what each element takes.

WRITTEN	WHAT IT IS
<code>page · component · store</code>	A declaration — a lowercase keyword names what is being made
<code>Button · Card · Table</code>	An element — capitalised, drawn from one registry
<code>.primary .lg .h1</code>	A flag — a boolean prop, or one case of an enum prop
<code>label: "Name"</code>	A prop — passed by name; one positional may lead
<code>Card { ... }</code>	A block — the element's children
<code>on click { ... }</code>	A handler — what happens when the reader acts
<code>style { padding: \$md }</code>	Raw CSS — with design tokens and live {values}

State is the only thing that moves.

What reads a state repaints when it changes. A derived value follows the values it reads; an effect runs when they do.



TYPES

The types a page needs, built in.

Each is a plain JSON value at run time, so it crosses a fetch, storage and the static paint unchanged.

Date

`@2026-03-14`

`.plus(days: 3)`

Time

`@09:30`

`.plus(minutes: 30)`

DateTime

`@2026-03-14T09:30Z`

`.inZone("Europe/Berlin")`

Duration

`90.minutes`

`.hours()`

Money

`€12.99`

`.convert(rate, "USD")`

Url

`"https://..."`

`.with(query: ...)`

Email

`"ada@example.com"`

`.domain()`

Color

`#0F766E`

`.mix(other, 0.2)`

Uuid

`uuid()`

a string's methods

File

`FileUpload`

`.preview()`

Number(1..=30)

`state nights = 12`

a condition, checked

Secret

`T12 if it escapes`

never shown, logged or kept

A mistake is found where it is written.

```
error[T05]: `User` has no field `nmae`  
  → src/pages/Profile.wf:3:18  
  |  
3 |   Heading(user.nmae).h1  
  |               ^^^^  
  = help: Its fields are `id`, `name`  
  = fix:  nmae → name
```

116 codes, one registry

The build, the editor, wf explain and the guide read the same list.

fix A fix where one is known

The nearest name, a ?., the missing props — applied by the editor.

1× Every mistake, once

Every file is read even when another does not parse.

02

What it builds

One program; five outputs. The same elements are a page in a browser and a page on paper.

Pick the output in one line of config.



Static site

Every page pre-rendered, then hydrated. Sitemap, canonical, Open Graph.

```
"ssg": true
```



Single-page app

A page per chunk, loaded when its route shows; a service worker on request.

```
"spa"
```



PDF

The page's own CSS, laid out on paper: running headers, contents, charts.

```
"pdf"
```



Slide deck

A slide to a page, from components and data — like this one.

```
"slides"
```



Custom elements

Components as tags for React, Vue, Rails or a plain page.

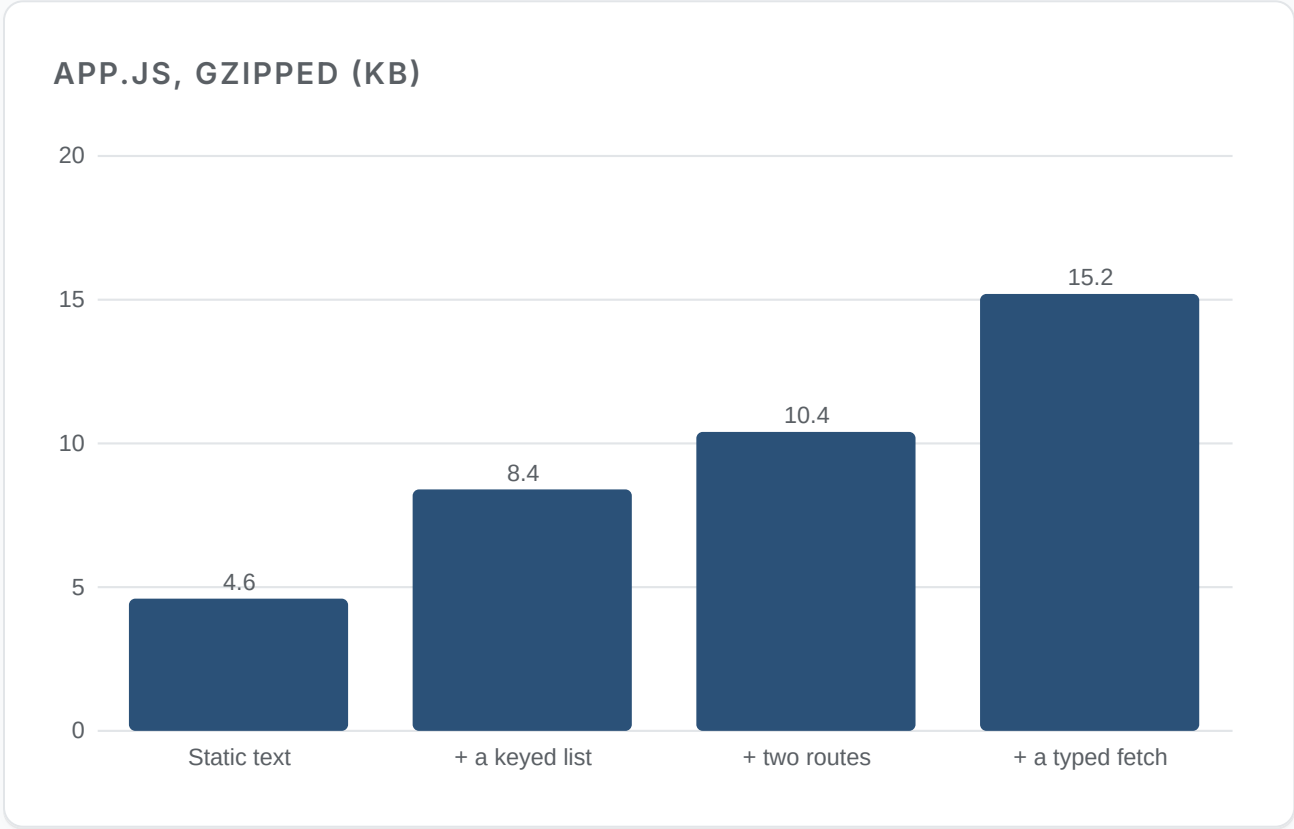
```
"elements"
```

webfluent.app.json

```
{ "build": { "output_type": "slides",  
             "slides": { "size": "16:9" } } }
```

The program does not change. A PDF is refused what only a browser can do — a button, a handler, a route — where it is written, and the editor offers only what the output draws.

The runtime holds only what a program reaches.

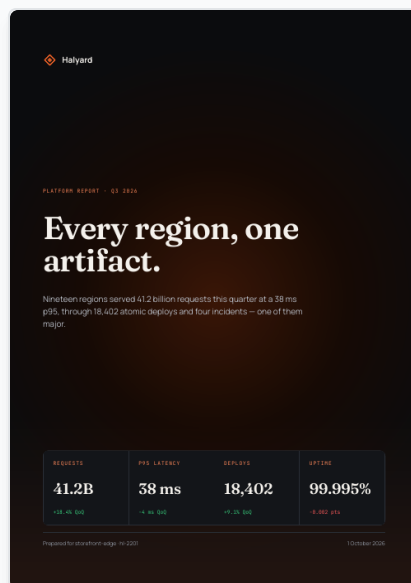


Static text	4.6 kB
+ a keyed list	8.4 kB
+ two routes	10.4 kB
+ a typed fetch	15.2 kB

Measured on four programs built with wf build. A for brings the keyed-list module, a second page the router, a resource the network engine.

The same CSS, on paper.

No browser, no external tool: the compiler lays out the page's HTML and CSS itself — the cascade, flex and grid, shaped Arabic, tables that repeat their header.



Report cover — a radial glow and stat tiles



Charts side by side, a table by content

Report: Performance report 20 2020

Workflow: Page 1 of 3

Deployments

The party four most recent deploy to production. A deploy is one content-addressed build reaching every region at once, or once in a deploy of an older hash.

Entity	Content	Source	Author	Time	Status	Web
1681888	Move image service to workers	edge-cache	benjam	64s	🟢 Full test pass	2020-09-28 12:27
1681834	Move image service to workers	edge-cache	margaret	77s	🔴 Full test	2020-09-28 13:18
1681829	Move localized build customer	edge-cache	and	99s	🔴 Full test	2020-09-28 17:05
1724474	Split checkout bundle	fr-edge-cache	benjam	176s	🟢 Full test	2020-09-29 20:46
1702161	Route DC:PRM to FR	edge-cache	and	188s	🟢 Full test	2020-09-29 14:14
7021275	Route DC:PRM to FR	edge-cache	margaret	109s	🟢 Full test	2020-09-29 18:53
1477704	Split checkout bundle	fr-edge-cache	benjam	144s	🟢 Full test	2020-09-28 14:29
1321437	Move image service to workers	fr-edge-cache	benj	144s	🟢 Full test	2020-09-28 18:17
4750428	Split image service parcel	checkout-v2	benjam	142s	🟢 Full test	2020-09-28 15:13
1350618	Fix TUI options for PCJ	edge-cache	benjam	140s	🔴 Full test	2020-09-27 18:45
7494121	Upgrade cache keys to http/ctrl	edge-cache	benjam	144s	🔴 Full test pass	2020-09-27 14:58
1725275	Split checkout bundle	checkout-v2	and	188s	🟢 Full test	2020-09-27 14:42
4864764	Split checkout bundle	edge-cache	benjam	118s	🟢 Full test	2020-09-26 20:59
7493248	Share rules between key format	key-formatting	benjam	112s	🟢 Full test	2020-09-26 12:18
4987034	Share rules between key format	key-formatting	benj	107s	🟢 Full test	2020-09-26 12:38
4864765	Upgrade HTTP to the edge	edge-cache	and	106s	🟢 Full test	2020-09-26 13:49
4864763	Upgrade HTTP to the edge	edge-cache	and	121s	🟢 Full test	2020-09-26 13:51
4645202	Split image service parcel	key-formatting	margaret	64s	🔴 Full test	2020-09-25 12:44
8075500	Share rules between key format	fr-edge-cache	margaret	88s	🟢 Full test	2020-09-24 16:58
1417100	Split image service parcel	checkout-v2	and	87s	🟢 Full test	2020-09-24 14:23
1288148	Upgrade checkout bundle	checkout-v2	benj	47s	🟢 Full test	2020-09-24 09:58

Fetch data from the report base. Not a new product.

Report: Page 1 of 3

64 rows, the header on every page



Meridian Studio

Product design and engineering

Invoice

No. 2024-0417

FROM

Meridian Studio

180 Hudson Ave, Suite 4

Portland, OR 97209

ENR-6-02557

BILL TO

Northwest Outfitters, Inc.

44N. Accounts Payable

88 Marston Drive, 6th floor

New York, NY 10012

Issued

Terms

Due

Sep 30, 2026

New 30

96-55332

Oct 30, 2026

#	DESCRIPTION	QTY	UNIT	PRICE	AMOUNT
1	Component library (Prpns) Change request QR-1	2	component	\$160.00	\$320.00
2	Illustration set Illustr 1.1	1	illustration	\$330.00	\$330.00
3	Performance budget review Illustr 1.1	1	day	\$1,100.00	\$1,100.00
4	Copy editing Change request QR-1	1	hour	\$65.00	\$65.00
5	Wireframes, checkout flow Illustr 1.1	1	screen	\$220.00	\$220.00
6	Front-end build, product pages Illustr 1.1	8	day	\$1,180.00	\$9,440.00
7	Wireframes, checkout flow Change request QR-1	1	screen	\$220.00	\$220.00
8	Front-end build, product pages Change request QR-1	3	day	\$1,180.00	\$3,540.00
9	UX research interviews Phase 2	3	session	\$580.00	\$1,740.00
10	Illustration set Change request QR-1	1	illustration	\$330.00	\$330.00
11	Copy editing Illustr 1.1	8	hour	\$85.00	\$680.00
12	Wireframes, checkout flow Change request QR-1	1	screen	\$220.00	\$220.00

Meridian Studio, 180 Hudson Ave, Portland, OR 97209 | 503-639-0000 | [meridianstudio.com](#)

Invoice 2024-0417 | page 1 of 1

An invoice: band, watermark, totals kept whole



Arabic, right to left, from the same engine

03

The tools

One binary: a compiler, a formatter, a test runner, a browser check, an audit — and a language server that knows what you are building.

Everything a project needs is a subcommand.

wf check	Every finding, with a code, a place and a fix; nothing written	wf test	test <code>"..." { }</code> blocks — snapshots, and clicks in a real Chrome
wf verify	Every route loaded in a headless browser: errors, paint, bytes	wf audit	What the project trusts: markup, origins, storage, env
wf fmt	One layout for every file — <code>.wf</code> braces or <code>.wfx</code> indents	wf docs	A gallery of every built-in and the project's own components
wf serve	Rebuild on save; a failed build drawn over the page	wf-lsp	Completion, hover, rename, quick fixes — for this output

In numbers.

79 built-in elements, from Button to TableOfContents	116 diagnostic codes, each with a page in the guide	37 runtime modules, shipped only when reached	48 chapters of the guide, every example built by the tests
5 outputs from one program	11 types the language brings: Date to Secret	32 icons, drawn inline, on screen and paper	1 binary: compiler, formatter, tests, server

This deck is a .wf file.

```
examples/decks/webfLuent/src/Deck.wf

component Opener(_ title: String, num: String, line: String) {
  Slide(class: "section") {
    Text(num, class: "section__num")
    Text(title, class: "section__title")
    Text(line, class: "section__line")
  }
}

page Deck(path: "/", title: "WebFluent") {
  Presentation {
    Opener("The language", num: "01",
      line: "One lexeme, one meaning.")
    for t in TOPICS { Topic(t) }
  }
}
```

Slides are components: an opener, a heading, a console frame — written once, placed sixteen times. A for makes a slide from each item of a list.

The fonts are files beside it; the colours are the docs site's tokens; wf build writes the PDF.



THE SOURCE OF EVERY DECK

GET STARTED

Write the page.

Let the compiler do the rest.

```
$ curl -sSL .../install.sh | bash
```

```
$ wf init my-site --template static
```

```
$ wf serve
```



webfluent.monzeromer.dev